

ICS-344

DVSA Vulnerability Discovery & Remediation

10 Official Vulnerabilities · AWS Serverless Security · KFUPM — ICS-344 Term 252

10

Vulnerabilities

8

AWS Services

100%

Fixed & Verified

Serverless Architecture Overview



Key AWS Services Involved

API Gateway	Public entry point — routes requests to Lambda	Amazon S3	Static frontend & order receipt storage
AWS Lambda	Serverless compute — executes business logic	IAM Roles	Execution permissions for Lambda functions
Amazon Cognito	User authentication & JWT token issuance	CloudWatch	Monitoring, logging & exploit verification
Amazon DynamoDB	NoSQL persistent storage for orders/users	SQS	Message queue for async order processing

10 Official Vulnerabilities — At a Glance

V1 Event Injection

RCE

V2 Broken Authentication

Auth

V3 Sensitive Info Disclosure

Access

V4 Insecure Cloud Config

Config

V5 Broken Access Control

Access

V6 Denial of Service

Avail.

V7 Over-Privileged Function

IAM

V8 Logic Vulnerability

Logic

V9 Vulnerable Dependencies

RCE

V10 Unhandled Exceptions

Info Leak

V1

Event Injection / Code Injection

Summary

A normal user can inject JavaScript code into the request body. The DVSA-ORDER-MANAGER Lambda uses node-serialize for deserialization, causing attacker-controlled input to execute inside the function.

Root Cause

node-serialize supports a `$$ND_FUNC$$` marker that automatically evaluates embedded functions. The backend called `serialize.unserialize(event.body)` directly on untrusted input — treating data as executable code.

Deviation Class

Intentional misuse / Unsafe deserialization / RCE

Fix Applied

Removed node-serialize entirely. Replaced with `JSON.parse(event.body)`. Added an allowlist validation for the 'action' field. Wrapped parsing in try/catch. Raw headers accessed via `event.headers` directly.

Code / Config Change

BEFORE: `serialize.unserialize(event.body)`

AFTER:

```
req = JSON.parse(event.body)
// + allowlist check on req.action
if (!allowedActions.includes(action))
    return 403 Unauthorized action
```

Post-Fix Verification

Same malicious `$$ND_FUNC$$` payload now returns `{status:'err', msg:'Unauthorized action'}`. CloudWatch shows no injected error — confirming code execution is fully blocked.

V2

Broken Authentication

Summary

A normal user (User B) can modify their JWT payload to replace username/sub with another user's identity (User C). The backend trusted the decoded payload without verifying the cryptographic signature.

Root Cause

The backend decoded the JWT payload using base64 and used token.username directly — without verifying the signature. Decoding is NOT the same as verifying. Any attacker can alter payload claims freely.

Deviation Class

Intentional misuse / JWT tampering / Broken auth

Fix Applied

Enabled the DVSA-CognitoAuthorizer on the /order POST method in API Gateway. API Gateway now verifies the JWT signature before forwarding to Lambda. Redeployed the dvsa stage.

Code / Config Change

```
API Gateway → /order → POST  
→ Method Request  
→ Authorization:  
    DVSA-CognitoAuthorizer
```

```
Then: Deploy API → Stage: dvsa
```

Post-Fix Verification

Forged token (User B payload with User C identity) now returns {message:'Unauthorized'}. Valid User B token still returns correct orders — legitimate access unaffected.

Sensitive Information Disclosure

Summary

Privileged admin receipt functionality (DVSA-ADMIN-GET-RECEIPT) can be accessed by a normal user to generate S3 signed URLs for other users' receipts — bypassing the intended authorization boundary.

Root Cause

Weak access control around the admin receipt-generation endpoint. The function did not validate that the requesting user is an admin or owns the requested order-id, allowing any authenticated user to obtain signed URLs for sensitive receipts.

Deviation Class

Intentional misuse / Broken access control / Info disclosure

Fix Applied

Enforce strict authorization on receipt-generation functions. Validate isAdmin flag on the server side. Restrict signed URL generation to admin roles only. Reduce S3 bucket exposure by scoping object-level access.

Code / Config Change

BEFORE: No admin check on receipt action

AFTER:

```
if (isAdmin !== "true") {
  return { statusCode: 403,
    body: JSON.stringify({
      status: "err", msg: "Unauthorized"
    })
  }
}
```

Post-Fix Verification

Normal users can no longer generate signed URLs for other users' receipts. Admin operations require verified admin credentials. Unauthorized access returns 403/Unauthorized.

Insecure Cloud Configuration

Summary

The DVSA S3 receipts bucket had Block Public Access disabled and write permissions open to any AWS account. Any external AWS user could upload arbitrary files to the bucket without restriction.

Root Cause

Improper S3 bucket configuration: Block Public Access was OFF, and bucket policy allowed external accounts write access. This violates the principle that cloud storage must not allow public write access.

Deviation Class

Misconfiguration / Insecure cloud storage

Fix Applied

Enabled all four Block Public Access settings on the S3 bucket. Removed public write permissions. Restricted PutObject access to specific trusted IAM roles only.

Code / Config Change

BEFORE: BlockPublicAccess: OFF

AFTER (applied via AWS Console):

```
{
  "BlockPublicAcls": true,
  "IgnorePublicAcls": true,
  "BlockPublicPolicy": true,
  "RestrictPublicBuckets": true
}
```

Post-Fix Verification

Same aws s3 cp command now returns:
"AccessDenied when calling PutObject"
Only authorized internal services can upload. Bucket is no longer publicly writable.

Broken Access Control

Summary

A normal authenticated user can call the 'complete' action through the public /order API, which routes directly to DVSA-ORDER-COMPLETE without any admin check — bypassing the intended payment workflow.

Root Cause

The 'complete' case in DVSA-ORDER-MANAGER routed directly to the DVSA-ORDER-COMPLETE function without checking user role. No ownership or admin validation was enforced, exposing a privileged workflow to regular users.

Deviation Class

Intentional misuse / Missing authorization / Broken access control

Fix Applied

Added an isAdmin check inside the 'complete' case. Non-admin requests immediately return 403 Unauthorized. Admin requests include user context in the payload for downstream validation.

Code / Config Change

```
BEFORE: case "complete":  
    payload = {orderId: req["order-id"]};
```

```
AFTER:  
if (isAdmin == "true") {  
    payload = {user, orderId, isAdmin};  
} else { return 403 Unauthorized; }
```

Post-Fix Verification

Complete action by normal user now returns:
{status:"err", msg:"Unauthorized"}
Normal "orders" action still works correctly for authenticated users.

Denial of Service (Billing Exhaustion)

Summary

Sending 500 concurrent billing requests saturates the Lambda concurrency limit and API Gateway burst limits. Legitimate users receive 429/5xx responses and cannot complete payment during the attack window.

Root Cause

The billing endpoint lacked rate limiting and concurrency isolation. A single authenticated attacker could exhaust shared Lambda capacity, causing TooManyRequestsException and service unavailability for other users.

Deviation Class

Intentional misuse / Resource exhaustion / DoS

Fix Applied

Implemented per-user rate limiting inside the Lambda function. Requests arriving within 1 second from the same user are rejected with HTTP 429. This limits blast radius per user and protects shared capacity.

Code / Config Change

```
user = event.get("authorization")
now = time.time()
if user in last_request_time:
    if now - last_request_time[user] < 1:
        return { "statusCode": 429,
                  "body": "Too many requests" }
last_request_time[user] = now
```

Post-Fix Verification

Flood attack now produces mix of HTTP 200 and HTTP 429 responses. System no longer returns 500/502 errors. Legitimate users remain able to complete billing normally.

Over-Privileged Function

Summary

The DVSA-ORDER-MANAGER Lambda was assigned a wildcard IAM policy (Action:*, Resource:*), granting unrestricted access to all AWS services. Compromising the function exposes full account-level permissions.

Root Cause

Improper IAM role configuration. The execution role had Action:* / Resource:* — violating least privilege. Any vulnerability that executes code inside Lambda automatically inherits these excessive permissions.

Deviation Class

Misconfiguration / Over-privileged IAM / Least privilege violation

Fix Applied

Removed wildcard permissions. Replaced with a restrictive policy allowing only lambda:InvokeFunction on specific function ARNs. Applied the Principle of Least Privilege.

Code / Config Change

BEFORE:

```
{"Effect": "Allow", "Action": "*", "Resource": "*"}
```

AFTER:

```
{"Effect": "Allow",  
  "Action": ["lambda:InvokeFunction"],  
  "Resource": [  
    "arn:aws:lambda:...DVSA-ORDER-NEW",  
    "arn:aws:lambda:...DVSA-ORDER-UPDATE"  
  ]  
}
```

Post-Fix Verification

IAM role now shows only specific Lambda ARN permissions. No wildcard entries remain. CloudTrail confirms the role can no longer access S3, DynamoDB, or other services outside its scope.

Logic Vulnerability (Race Condition)

Summary

A user could modify shipping details or send duplicate billing requests after an order was already paid. The shipping function only blocked updates at `orderStatus >= 200`, but status 120 already means 'paid'.

Root Cause

Business logic flaw: The threshold for blocking updates was set too late in the status workflow. Status 120 = paid, but the check used `>= 200`. This allowed concurrent billing+update requests to create inconsistent order states.

Deviation Class

Intentional misuse / Business logic flaw / Race condition

Fix Applied

Added `orderStatus >= 120` check in DVSA-ORDER-SHIPPING, DVSA-ORDER-UPDATE, and DVSA-ORDER-BILLING. Used DynamoDB `ConditionExpression` to prevent race conditions atomically.

Code / Config Change

BEFORE: `if orderStatus >= 200: reject`

AFTER:

```
if int(orderStatus) >= 120:
    return {"status": "err",
            "msg": "too late to update order"}
```

DynamoDB atomic guard:

```
ConditionExpression="orderStatus < :paid"
```

Post-Fix Verification

Duplicate billing returns: `{status:"err",msg:"order already made"}`

Post-payment shipping update returns: `{status:"err",msg:"too late to update order"}`

Normal order viewing still works.

Vulnerable Dependencies (node-serialize)

Summary

The DVSA-ORDER-MANAGER Lambda uses node-serialize, a package with a known critical vulnerability. The \$\$ND_FUNC\$\$ marker allows arbitrary JavaScript execution from user-controlled input — enabling full RCE.

Root Cause

Insecure third-party dependency. node-serialize extends JSON with function serialization capability. The \$\$ND_FUNC\$\$ marker causes automatic evaluation of embedded function bodies during deserialization.

Deviation Class

Vulnerable dependency / Supply chain risk / RCE

Fix Applied

Removed node-serialize from the Lambda function entirely. Replaced serialize.unserialize() with JSON.parse(). Added input validation and action allowlist. Performed dependency audit with npm audit.

Code / Config Change

BEFORE:

```
const serialize = require("node-serialize");  
var req = serialize.unserialize(event.body);
```

AFTER:

```
var req = JSON.parse(event.body);  
// node-serialize removed from package.json  
// npm audit shows 0 critical issues
```

Post-Fix Verification

Same \$\$ND_FUNC\$\$ payload no longer executes. CloudWatch shows no FILE READ SUCCESS or injected output. Error occurs before any attacker-controlled code runs — vulnerability fully mitigated.

V10

Unhandled Exceptions

Summary

Sending a request without the required order-id field causes DVSA-ORDER-GET to throw a Python KeyError. The raw exception — including file path /var/task/get_order.py, code line, and stack trace — is returned to the user.

Root Cause

The backend used event['orderId'] direct dictionary access without validation. Missing keys raise unhandled KeyError exceptions. No try/except wrapper was present, so raw error details were exposed to the client.

Deviation Class

Poor input validation / Information disclosure / Improper error handling

Fix Applied

Replaced direct dictionary access with event.get(). Added explicit presence checks for required fields. Wrapped all database logic in try/except. Returns only safe generic error messages to the client.

Code / Config Change

BEFORE:

```
orderId = event["orderId"] # KeyError!
```

AFTER:

```
orderId = event.get("orderId")
if not orderId:
    return {"status": "err",
            "msg": "Missing required field: orderId"}
```

Post-Fix Verification

Same malformed request now returns:

```
{status: "err", msg: "Missing required field: orderId"}
```

No stack trace, file path, or source code line is exposed. Internal errors logged to CloudWatch only.

Structured Security Analysis — Table A

Intended Rules vs. Exploit Behavior

Vulnerability	Intended Rule	Normal Behavior	Exploit Behavior
V1 — Event Injection	Input treated as data only	Valid actions routed correctly	\$\$ND_FUNC\$\$ payload executed in Lambda
V2 — Broken Auth	JWT signature must be verified	Own token → own orders only	Forged token returns victim's orders
V3 — Info Disclosure	Admin-only receipt generation	User accesses own receipts	Signed URLs generated for any order
V4 — Insecure Config	No public write to S3	Upload restricted to system	Any AWS account could upload files
V5 — Broken Access	Normal user cannot complete order	User views own orders	Complete action reached by normal user
V6 — DoS	Fair rate-limited billing	Single request returns 200	500 concurrent requests exhaust Lambda
V7 — Over-Privilege	Lambda has minimal IAM perms	Only needed services accessed	Action:* Resource:* grants full AWS access
V8 — Logic Flaw	No modifications after payment	User can update pre-payment	Shipping changed after billing completed
V9 — Vulnerable Dep.	Input never executed as code	Actions processed as JSON data	node-serialize executes function payload
V10 — Unhandled Exc.	Safe error messages only	Validation error returned safely	Stack trace + file path exposed to user

Structured Security Analysis — Table B

Deviation Class · Fix Applied · Post-Fix Verification

Vulnerability	Why It Deviates	Class	Fix	Verified
V1 — Event Injection	User input executed as code	Misuse/RCE	JSON.parse + allowlist	Unauthorized action returned
V2 — Broken Auth	Unsigned JWT payload trusted	Misuse/Auth	Cognito Authorizer on API GW	Forged token → Unauthorized
V3 — Info Disclosure	No admin check on receipts	Misuse/Access	isAdmin server-side check	Normal users blocked
V4 — Insecure Config	Public S3 write allowed	Misconfiguration	Block Public Access ON	Upload → AccessDenied
V5 — Broken Access	Privileged action exposed publicly	Misuse/Access	isAdmin check in complete case	Normal user → Unauthorized
V6 — DoS	No rate limiting on billing	Misuse/Avail.	Per-user rate limiting in Lambda	429 returned on excess requests
V7 — Over-Privilege	IAM wildcard permissions	Misconfiguration	Least-privilege IAM policy	No wildcard perms remain
V8 — Logic Flaw	Updates allowed post-payment	Misuse/Logic	orderStatus >= 120 checks + ConditionExpr.	Order already made error
V9 — Vulnerable Dep.	Unsafe deserialization library	Supply chain/RCE	Removed node-serialize	No code execution in CloudWatch
V10 — Unhandled Exc.	Raw stack trace returned to user	Info Disclosure	try/except + event.get()	Safe error message only

Key Security Takeaways

01 Never Trust Unverified Input

Deserializing untrusted input can turn a simple API call into remote code execution. Always parse with `JSON.parse` and validate with allowlists.

02 JWT Decoding $\neq$ JWT Verification

Decoding a token reads its claims. Verifying checks the cryptographic signature. Only verified tokens should determine user identity.

03 Least Privilege for IAM Roles

Wildcard permissions (`Action:*` `Resource:*`) turn every Lambda vulnerability into full AWS account compromise. Scope roles to exact ARNs needed.

04 Business Logic Must Live in Backend

Frontend checks are trivially bypassed. Order workflow state transitions must be enforced server-side using conditional database updates.

05 Rate Limiting Protects Availability

Serverless $\neq$ unlimited. Without rate limiting, a single attacker can exhaust concurrency and deny service to all users.

06 Never Expose Raw Exceptions

Stack traces, file paths, and code lines are a roadmap for attackers. Always catch exceptions and return only safe, generic error messages.

ICS-344

DVSA Vulnerability Discovery & Remediation

10 Vulnerabilities · Fully Exploited · Fixed · Verified

Thank You

KFUPM — ICS-344 Term 252